

SENSITIVE INFORMATION IN REPR STRINGS

As mentioned earlier, we usually display the `str` string to users, and we use the `repr` string in technical contexts, such as logfiles. But the `repr` string can cause security issues if the object you're creating contains sensitive information, such as passwords, medical details, or personally identifiable information. If this is the case, make sure the `__repr__()` method doesn't include this information in the string it returns. When software crashes, it's frequently set up to include the contents of variables in a logfile to aid in debugging. Often, these logfiles aren't treated as sensitive information. In several security incidents, publicly shared logfiles have inadvertently included passwords, credit card numbers, home addresses, and other sensitive information. Keep this in mind when you're writing `__repr__()` methods for your class.

Numeric Dunder Methods

The *numeric dunder methods*, also called the *math dunder methods*, overload Python's mathematical operators, such as `+`, `-`, `*`, `/`, and so on. Currently, we can't perform an operation like adding two `WizCoin` objects together with the `+` operator. If we try to do so, Python will raise a `TypeError` exception, because it doesn't know how to add `WizCoin` objects. To see this error, enter the following into the interactive shell:

```
>>> import wizcoin
>>> purse = wizcoin.WizCoin(2, 5, 10)
>>> tipJar = wizcoin.WizCoin(0, 0, 37)
>>> purse + tipJar
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'WizCoin' and 'WizCoin'
```

Instead of writing an `addWizCoin()` method for the `WizCoin` class, you can use the `__add__()` dunder method so `WizCoin` objects work with the `+` operator. Add the following to the end of the `wizcoin.py` file:

```
--snip--
❶ def __add__(self, other):
    """Adds the coin amounts in two WizCoin objects together."""
❷ if not isinstance(other, WizCoin):
    return NotImplemented
```